Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores

CE-1107 Fundamentos de Arquitectura de Computadores

Bitácora

Proyecto Grupal 1: Máquinas de estados

Sistema de puerta con contraseña

Profesor: Luis Chavarría Zamora

Integrantes:

Isaac Andrés Arias Picado
Byron Josué Bolaños Porras
Jose Andrés Chavarría Gamboa
Isac Marín Sirias

II Semestre 2026

Índice

1. Entradas **2**
18 de octubre de 2026 2
19 de octubre de 2026 3
21 de octubre de 2026 3
23 de octubre de 2026 5
24 de octubre de 2026 6
27 de octubre de 2026 8

1. Entradas

18 de octubre de 2026

Integrante(s): Isaac Andrés Arias Picado

Tareas realizadas

Se creó el repositorio con la propuesta de implementación, el proyecto de Quartus y el archivo `.gitignore`. En la rama `setup/inicial` se implementó el paquete `config_pkg`, que reúne todas las constantes del sistema (frecuencias, tiempos, códigos de estados y eventos) para que ningún otro módulo tenga números fijos. Este cambio se integró con el pull request #1. Después, en la rama del bloque A, se implementaron los módulos `clk_enables` y `debounce` con sus testbenches y el script `run.do` que corre todas las simulaciones.

Módulo clk_enables

Genera pulsos de un ciclo a partir del reloj de 50 MHz para que los demás módulos trabajen a frecuencias bajas sin dividir el reloj. Produce un pulso de captura cada 2 s y un pulso de 1 kHz que comparten el debounce y el display. Usar un solo contador para estos dos ahorra flip-flops. El reset entra por la habilitación del contador y no por una entrada asíncrona.

$$
\begin{aligned}
CAP_BITS = \lceil \log_2(100\,000\,000) \rceil = 27, \quad SHARED_BITS = \lceil \log_2(50\,000) \rceil = 16 \\
tick_capture = (cap_cnt = 99\,999\,999) \wedge \neg reset \\
tick_debounce = tick_display = (shared_cnt = 49\,999) \wedge \neg reset \\
en_{cap} = \neg reset \wedge \neg is_max_{cap} \\
cnt_{next} = en \wedge (cnt + 1)
\end{aligned}
$$

Módulo debounce

Los botones solo pasan por dos flip-flops de sincronización y un detector de flanco, ya que la FPGA tiene Schmitt trigger. El detector de dedos viene de hardware externo sin Schmitt trigger, por lo que se filtra exigiendo 20 muestras iguales seguidas a 1 kHz. Luego se valida que haya como máximo un dedo activo y se codifica a un símbolo de 0 a 5.

$$
\begin{aligned}
finger_valid = (stable_0 + stable_1 + stable_2 + stable_3 + stable_4 \leq 1) \\
sym_0 = stable_0 \vee stable_2 \vee stable_4 \\
sym_1 = stable_1 \vee stable_2 \\
sym_2 = stable_3 \vee stable_4
\end{aligned}
$$

Cuadro 1: Filtro de estabilidad del detector de dedos (cada tick de 1 kHz)

Condición	**candidate**	**count**	**stable**
reset	00000	0	00000
raw $\neq$ candidate	raw	1	Sin cambio
raw = candidate y count $<$ 19	Sin cambio	count + 1	Sin cambio
raw = candidate y count = 19	Sin cambio	20	candidate

Verificación

El testbench de `clk_enables` pasó sus 4 pruebas (período de captura, período compartido, reset y primer tick). El de `debounce` pasó sus 9 pruebas, que cubren el pulso único de cada botón, el filtro de 20 muestras y la validación de patrones con dos o más dedos.

19 de octubre de 2026

Integrante(s): Isaac Andrés Arias Picado

Tareas realizadas

Se implementó el módulo `finger_capture` con su testbench, se documentó el bloque A y se integró a la rama `development` con el pull request #2.

Módulo finger_capture

Mantiene una ventana con los últimos 3 símbolos capturados, de 3 bits cada uno. En cada pulso de captura entra el símbolo nuevo y sale el más viejo. Los símbolos inválidos se ignoran, para que una lectura errónea no desplace un símbolo bueno.

$$
\begin{aligned}
clear_all &= reset \lor clear_capture \\
shift_en &= tick_capture \land finger_valid \land \neg clear_all \\
mask_hold &= \neg shift_en \land \neg clear_all \\
window_{next} &= (shift_en \land \{window[5:0], sym\}) \lor (mask_hold \land window)
\end{aligned}
$$

Cuadro 2: Ejemplo de llenado de la ventana

Tick	Símbolo	Válido	Ventana
1	001	1	000 000 001
2	010	1	000 001 010
3	011	1	001 010 011
4	100	1	010 011 100
5	100	0	010 011 100 (sin cambio)

Verificación

El testbench pasó sus 6 pruebas, que revisan el reset, la limpieza, el desplazamiento, que un símbolo inválido no cambie la ventana y el orden correcto tras cuatro capturas.

21 de octubre de 2026

Integrante(s): Isac Marín Sirias

Tareas realizadas

Se implementaron los cinco módulos del bloque D, encargado de las salidas del sistema, junto con sus testbenches y su documentación.

Módulo seven_seg

Convierte un código de 5 bits en los siete segmentos de un display, activos en bajo. Para cada carácter se genera un mintérmino con el AND de los cinco bits de entrada, y cada segmento es el NOT del OR de los mintérminos de los caracteres que lo encienden. Además de 0 a F, muestra las letras o, c, n y t para los estados y modos, y los códigos 20 a 31 dejan el display apagado.

$$character_1 = \neg c_4 \wedge \neg c_3 \wedge \neg c_2 \wedge \neg c_1 \wedge c_0$$
$$seg_a = \neg(character_0 \vee character_2 \vee character_3 \vee \ldots \vee character_{15})$$

Módulo pwm

Genera una señal de 1 kHz con un ciclo de trabajo de $s/7$. Un contador recorre los 50 000 ciclos de cada período, y la salida está en alto mientras el contador es menor que un umbral que depende de la velocidad. El umbral se elige con máscaras y la velocidad nueva solo se toma al empezar un período.

Cuadro 3: Umbral del PWM por velocidad

Velocidad	Ciclos en alto	Duty
0	0	0 %
1	7143	14.3 %
2	14286	28.6 %
3	21429	42.9 %
4	28571	57.1 %
5	35714	71.4 %
6	42857	85.7 %
7	50000	100 %

$$pwm_out = running \wedge \neg reset \wedge (counter < threshold)$$
$$threshold = \bigvee_{s=1}^{7} \big((active_speed = s) \wedge H(s)\big)$$

Módulo motor_ctrl

Es combinacional y dirige el PWM a una de las dos líneas del puente H según la dirección. Las dos salidas nunca pueden estar activas a la vez, porque una exige $dir_1 \wedge \neg dir_0$ y la otra $\neg dir_1 \wedge dir_0$.

$$line_{abrir} = \neg reset \wedge dir_1 \wedge \neg dir_0 \wedge pwm$$
$$line_{cerrar} = \neg reset \wedge \neg dir_1 \wedge dir_0 \wedge pwm$$

Módulo buzzer_ctrl

Genera una onda cuadrada para un buzzer pasivo. La frecuencia define el tono y un contador de duración define el patrón de cada evento. Los sonidos continuos se mantienen mientras el código esté activo, y los finitos se inician con un pulso y terminan solos. Un sonido continuo cancela uno finito.

Cuadro 4: Eventos del buzzer

Código	Evento	Frecuencia	Patrón
1	TEST	1000 Hz	Continuo
2	REG_OK	1000 Hz	Un beep de 100 ms
3	VALID	1000 Hz	Dos beeps de 100 ms
4	INVALID	500 Hz	Un beep de 500 ms
5	OPENING	2000 Hz	Continuo
6	CLOSING	1500 Hz	Continuo

Módulo display_ctrl

Selecciona el carácter de cada uno de los seis displays y usa una instancia de `seven_seg` por display, sin barrido. HEX0 muestra el estado de la puerta, HEX1 a HEX3 los tres dígitos capturados, HEX4 el modo y HEX5 la velocidad. En administración y prueba, HEX0 muestra la posición recordada de la puerta.

Verificación

Se prepararon testbenches para los cinco módulos. Cada uno compara las salidas contra una tabla de referencia independiente, por ejemplo las 32 entradas del decodificador o las 16 combinaciones del control del motor. Según la documentación del bloque, la ejecución de estas simulaciones queda pendiente.

23 de octubre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Tareas realizadas

Se revisó el estado general del repositorio y cada una de sus ramas ya hechas hasta el momento para comparar el enunciado con el archivo de propuesta en la rama principal. Se analizó cómo adecuar el bloque C (encargado de la máquina de estados principal e integración con el sistema) de trabajo a la implementación ya realizada en los bloques A y D, esto porque la máquina de estados principal debe poder conectarse correctamente a estas.

Procedimiento y decisiones

Al comparar el enunciado con el trabajo realizado se encontraron varios puntos importantes a resolver. En primer lugar se identificó que no hay forma de verificar cuándo una contraseña es inválida o no ya que la comparación se realiza de manera continua. Además, como el patrón de sin poner ningún dedo se establece como un cero, constantemente la ventana se llena de ceros sin requerir interacción con el sistema. Se plantea cambiar el sistema por intentos y un contador, así al completar el intento se evalúa y se limpia la captura.

Se realizó también una reestructuración del árbol de commits para ajustarse a lo que pide el enunciado (una nueva rama de trabajo por cada nueva implementación creada desde la rama development). Este cambio se integró mediante pull request y los cambios y fechas quedaron intactos.

24 de octubre de 2026

Integrante(s): Jose Andrés Chavarría Gamboa

Módulo timer_travel

Se implementó el contador que usa la máquina de estados para medir tiempos. Cuenta milisegundos usando el tick de 1 kHz que ya genera el bloque A, así el registro necesita pocos bits. La primera versión tenía una salida de espera de 1 s y un timeout de 30 s, pensando en usar finales de carrera para detectar la llegada de la puerta. Más tarde se decidió detectar el fin de carrera por tiempo, por lo que el timeout se reemplazó por una tabla de tiempos de viaje según la velocidad.

El tiempo de viaje sigue la fórmula de la propuesta y se guarda en milisegundos en el paquete de configuración.

$$
\begin{aligned}
T(s) &= \frac{7 \cdot T_{max}}{s}, \quad T_{max} = 4000 \text{ ms} \\
&\Rightarrow T(1) = 28000,\ T(2) = 14000,\ \ldots,\ T(7) = 4000 \text{ ms} \\
W &= \lceil \log_2(28000 + 1) \rceil = \lceil 14.77 \rceil = 15 \text{ bits}
\end{aligned}
$$

La tabla se implementó con máscaras, comparando la velocidad con cada valor y dejando pasar solo el tiempo correspondiente, de la misma forma que el umbral del PWM. El contador se detiene en el viaje más largo para no desbordarse y para que la espera de 1 s siga funcionando cuando la velocidad es 0.

$$
\begin{aligned}
clr &= reset \vee start \\
inc &= tick_ms \wedge \neg clr \wedge \neg at_max \\
hold &= \neg clr \wedge \neg inc \\
count_{next} &= (inc \wedge (count + 1)) \vee (hold \wedge count) \\
limit &= \textstyle\bigvee_{k=1}^{7} \big((speed = k) \wedge T(k)\big) \\
wait_done &= (count \geq 1000) \\
travel_done &= (count \geq limit)
\end{aligned}
$$

Módulo attempt_ctrl

Se creó un módulo nuevo que detecta cuándo termina un intento de contraseña. Sin él, la comparación sería continua y nunca habría un momento para decidir que una contraseña es inválida. El intento empieza con el primer dígito distinto de 0, ya que el 0 es lo que se captura cuando nadie pone los dedos, y termina al capturar el tercer dígito. La salida ready se registra un ciclo después, porque en ese mismo flanco la ventana del bloque A guarda el tercer dígito.

$$
\begin{aligned}
clr &= reset \lor clear_capture \\
shift &= tick_capture \land finger_valid \\
nz &= sym_2 \lor sym_1 \lor sym_0 \\
start &= shift \land nz \land (cnt = 0) \land \neg clr \\
adv &= shift \land (cnt = 1) \land \neg clr \\
fin &= shift \land (cnt = 2) \land \neg clr \\
hold &= \neg clr \land \neg start \land \neg adv \land \neg fin \\
cnt_{next} &= (start \land 01) \lor (adv \land 10) \lor (hold \land cnt) \\
ready_{next} &= fin
\end{aligned}
$$

Diseño de la máquina de estados

Se rediseñó la tabla de transiciones de la propuesta para que cumpla el enunciado. Las dos contraseñas guardadas pasan a ser equivalentes, cualquiera abre la puerta si está cerrada y la cierra si está abierta. En administración el submodo escoge el espacio de memoria, y si la contraseña ingresada ya existe se borra, si no se guarda. Se agregó el estado VALID, que espera 1 s antes de mover el motor, porque el sonido continuo de apertura cancelaría el sonido de contraseña válida. Además, el reset borra las contraseñas pero no detiene un movimiento en curso, para que la puerta no quede a medio camino y el sistema no pierda su posición.

Cuadro 5: Tabla de transiciones de la FSM principal

Estado	**Condición**	**Siguiente**	**Acción**
ERROR	rst	ERROR	Borrar contraseñas
ERROR	adm	ADMIN	mode $\leftarrow 1$
ERROR	tst y puerta cerrada	TEST	Fase 0
ERROR	ready	ERROR	INVALID
CLOSED/OPEN	rst	ERROR	Borrar contraseñas
CLOSED/OPEN	adm	ADMIN	mode $\leftarrow 1$
CLOSED	tst	TEST	Fase 0
CLOSED/OPEN	ready y match	VALID	Guardar velocidad
CLOSED/OPEN	ready y no match	Igual	INVALID
VALID	wait_done y speed $\neq 0$	OPENING/CLOSING	Arrancar motor
VALID	wait_done y speed $= 0$	CLOSED/OPEN	No se mueve
OPENING/CLOSING	rst	Igual	Borrar contraseñas, motor sigue
OPENING/CLOSING	travel_done	OPEN/CLOSED*	Motor apagado
ADMIN	rst	ERROR	Borrar contraseñas
ADMIN	mode $= 0$	CLOSED/OPEN*	Salir
ADMIN	adm	ADMIN	mode: $1 \rightarrow 2 \rightarrow 0$
ADMIN	ready y match	ADMIN	Borrar, REG_OK, mode $\leftarrow 0$
ADMIN	ready y no match	ADMIN	Guardar en espacio mode, REG_OK, mode $\leftarrow 0$
TEST	fase 0 y travel_done	TEST	Fase 1 (espera 1 s)
TEST	fase 1 y wait_done	TEST	Fase 2 (cerrar)
TEST	fase 2 y travel_done	CLOSED*	Fin

* Pasa a ERROR si no hay contraseñas registradas. Prioridad: rst > adm > tst > ready.

Verificación

El timer se verificó con un testbench de 14 pruebas, que revisa el tiempo de viaje de las siete velocidades justo en el tick límite, la saturación del contador, el caso de velocidad 0 y el reinicio con start y con reset. El módulo attempt_ctrl se verificó con 7 pruebas usando el módulo de captura real del bloque A, para confirmar que al llegar ready la ventana tiene los tres dígitos. Todas las pruebas pasaron.

27 de octubre de 2026

Integrante(s): Byron Josué Bolaños Porras

Tareas realizadas

Se implementaron los tres módulos del bloque B, encargado del cálculo de la velocidad y de la memoria de contraseñas, cada uno con su testbench y su documentación, en la rama `BloqueB-Datos`. El bloque se ajustó a las señales definidas para la máquina de estados principal, con dos contraseñas equivalentes y la operación de la ALU dada por el primer dígito de 1 a 5.

Módulo alu

Calcula la velocidad de la puerta con los tres dígitos de la ventana. El primer dígito elige la operación y los otros dos son los operandos A y B. El resultado es de 3 bits y circular, lo que da una velocidad de 0 a 7. No existe operación nula, ya que un intento nunca empieza con 0, y cualquier código fuera de 1 a 5 da 0. La multiplicación se arma con productos parciales sin usar el operador de multiplicar, y como girar 3 bits tres veces devuelve el mismo valor, las rotaciones usan B mód 3 giros.

Cuadro 6: Operaciones de la ALU

op	Operación	Resultado
1	Suma	$(A + B)$ mód 8
2	Resta	$(A - B)$ mód 8
3	Multiplicación	$(A \cdot B)$ mód 8
4	Rotación izquierda	$rot_{izq}(A,\ B \text{ mód } 3)$
5	Rotación derecha	$rot_{der}(A,\ B \text{ mód } 3)$

$$
\begin{aligned}
mult_0 &= A_0 \wedge B_0 \\
mult_1 &= (A_1 \wedge B_0) \oplus (A_0 \wedge B_1) \\
mult_2 &= (A_2 \wedge B_0) \oplus (A_1 \wedge B_1) \oplus (A_0 \wedge B_2) \oplus \big((A_1 \wedge B_0) \wedge (A_0 \wedge B_1)\big) \\
mod_1 &= (B = 1) \vee (B = 4) \vee (B = 7), \quad mod_2 = (B = 2) \vee (B = 5) \\
rotl &= (mod_0 \wedge A) \vee (mod_1 \wedge \{A_1 A_0 A_2\}) \vee (mod_2 \wedge \{A_0 A_2 A_1\}) \\
result &= (sel_{sum} \wedge sum) \vee (sel_{sub} \wedge sub) \vee (sel_{mult} \wedge mult) \vee (sel_{rotl} \wedge rotl) \vee (sel_{rotr} \wedge rotr)
\end{aligned}
$$

Módulo password_mem

Guarda hasta dos contraseñas en dos espacios equivalentes, cada uno con su bandera de validez. La máquina de estados escribe, borra o limpia mediante pulsos de un ciclo. En cada espacio el borrado tiene prioridad sobre la escritura, para que una escritura simultánea con un borrado nunca deje datos a medias.

Cuadro 7: Transición de cada espacio de memoria

kill	wr	pass siguiente	valid siguiente
1	X	000 000 000	0
0	1	window	1
0	0	Sin cambio	Sin cambio

$$
\begin{aligned}
kill_1 &= reset \vee clear_all \vee del_1 \\
load_1 &= wr_1 \wedge \neg kill_1 \\
hold_1 &= \neg wr_1 \wedge \neg kill_1 \\
pass1_{next} &= (load_1 \wedge window) \vee (hold_1 \wedge pass1) \\
valid1_{next} &= load_1 \vee (hold_1 \wedge valid1)
\end{aligned}
$$

Módulo comparator

Compara la ventana contra las dos contraseñas guardadas con un XOR bit a bit y un OR de reducción. Un espacio vacío nunca coincide, porque sin la bandera de validez una ventana en ceros igualaría una memoria vacía y daría un acierto falso.

$$equal_1 = \neg \bigvee_{i=0}^{8} (window_i \oplus pass1_i)$$
$$match_1 = equal_1 \wedge valid_1$$

Verificación

Los testbenches de la ALU, la memoria y el comparador pasaron sus 13, 15 y 11 pruebas. La ALU se probó con las 64 combinaciones de operandos para cada operación, la memoria con los casos de prioridad entre escritura y borrado, y el comparador con un barrido de las 512 ventanas posibles.